

Diagramas del sistema

DaVinci Paddle Boat · Reto EIA 2027 · Marymount

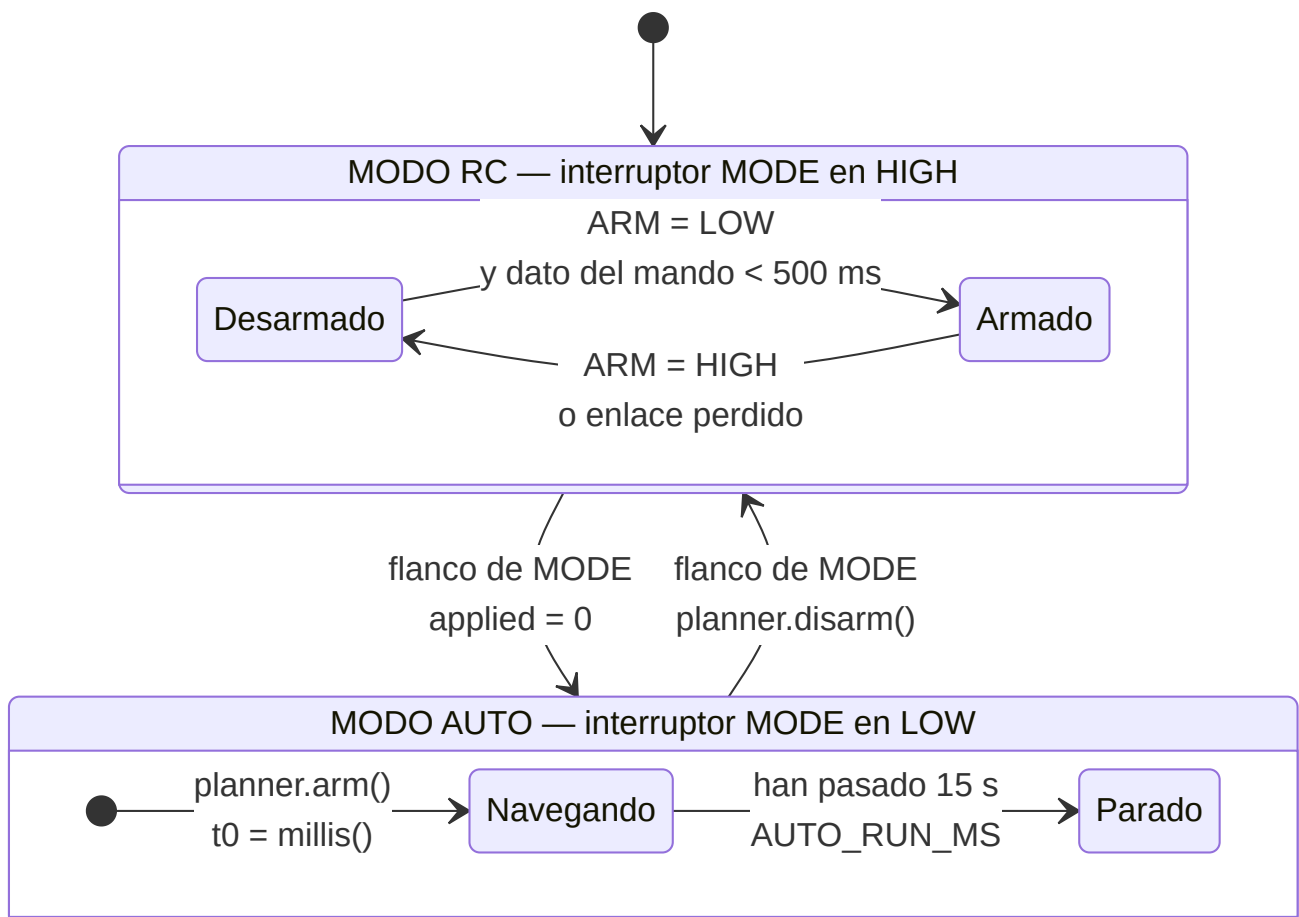
Por qué solo dos diagramas

El código de cada ejercicio son **60–99 líneas** con 1–4 TODO: cabe en una pantalla y se lee, no se dibuja. Diagramarlo enseñaría que el diagrama es lo que se mira y el código lo que se ignora. Lo que *sí* necesita dibujo es lo que no cabe en la cabeza: en qué estado está el barco, y la cadena entera del mando al agua.

Los dos salen de `diagramas/*.mmd` con `python3 build.py`, que rehace a la vez las imágenes, este PDF y la nota del Vault. **Fuente única.**

1. Los estados del barco

Lo más difícil de *Phase 5*, y un concepto de **seguridad**: mientras el barco no esté armado, el motor no gira pase lo que pase.



Hay **dos cosas independientes** que se confunden:

	Quién lo decide	Qué significa
Modo	el interruptor MODE (GPIO 26)	de dónde vienen las órdenes: mando o planificador
Armado	el interruptor ARM (GPIO 32) y el failsafe	si se les hace caso, o se manda 0

Se puede estar en AUTO y desarmado, y en RC y desarmado. **El modo no arma nada.**

El failsafe tiene dos gatillos

`applyFailsafe()` fuerza `armed = false` si se cumple **cualquiera**:

1. El interruptor ARM está abierto (lee HIGH).
2. La orden más reciente tiene más de `FAILSAFE_MS = 500 ms`.

El segundo es el que salva el barco: si el mando se apaga o se sale de rango, no sigue navegando con el último `throttle` recibido.

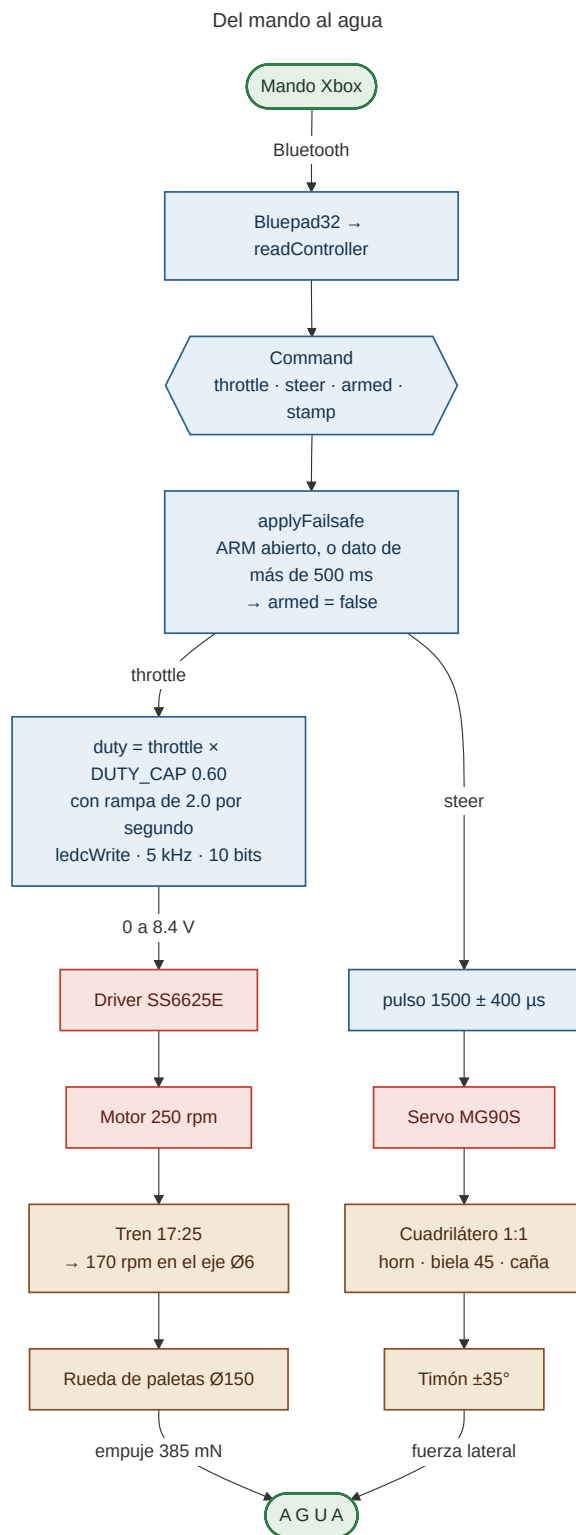
Detalle que se pasa por alto

En el flanco de cambio de modo, `applied` se pone a **0**: la rampa de aceleración empieza de cero. Sin eso, al saltar a AUTO el barco arrancaría de golpe con la velocidad que llevaba.

LED	Estado
Fijo	armado — el motor puede girar
Parpadeo lento (400 ms)	RC, desarmado
Parpadeo rápido (150 ms)	AUTO, desarmado o run terminado

2. Del mando al agua

Éste es el diagrama del proyecto: el único sitio donde el firmware y todo el trabajo mecánico aparecen juntos. Azul lo que se programa, rojo lo que se conecta, marrón lo que se corta y se imprime.



Lo que se ve aquí y no leyendo el código

- **Una sola estructura Command alimenta las dos ramas.** Propulsión y gobierno no son dos programas: son dos consumidores del mismo dato.
- **El failsafe está antes de la bifurcación**, así que apaga las dos cosas a la vez. No hay forma de quedarse con timón pero sin motor, ni al revés.
- **Cada flecha de la mitad de abajo es una pieza física** que hay que cortar, imprimir o pegar. La cadena mecánica es más larga que la de software.
- **Dónde se pierde y se gana.** El tren 17:25 baja de 250 a 170 rpm y a cambio multiplica el par por 1.47. El cuadrilátero del timón, en cambio, es **1:1 a propósito**: brazos iguales, para que calibrar el servo sea un número por lado y no una tabla.

Cómo usarlos en clase

1. **Antes de escribir código:** el diagrama 2 *tapando la mitad de abajo*. «¿Qué tiene que producir el firmware?» → un **Command**. Eso es todo.
2. **En Phase 5:** el diagrama 1, pero que dibujen ellos las transiciones antes de verlo. Casi siempre se les olvida que el failsafe tiene *dos* gatillos, no uno.
3. **Al final:** el diagrama 2 entero, para que vean que el **throttle** que escribieron acaba siendo empuje en el agua a través de once cosas.